Independent Opinion Review

ANLY656 Week 7 - Nominal Random Forest for Cellphone Activity

Reviewer	Composer 2.5, independent opinion review
Scope	Evaluation of the analysis that was produced. The solution was not rewritten, and the model was not re-run.
Materials	`wk7_solution.py`, `wk7_output.txt`. Assignment brief `Wk7_Assignment.pdf` and `Notes/Initial_Prompt.txt` used as context.
Date	15 October 2026

---

1. Interpretation of the results

The program classifies five activity labels from smartphone accelerometer-style readings (`sitting`, `sittingdown`, `standing`, `standingup`, `walking`) using `Data/CellphoneActivity.csv`. The file has **165,633** rows and **19** columns. The target is nominal `activity` with five classes. After `ReplaceImputeEncode`, the modeling table has **165,633** cases and **21** encoded columns; RIE reports **16 interval**, **1 binary**, **1 nominal**, and **1 excluded** attribute, with **zero** missing values and **zero** outliers on every listed field.

The data map in code types `activity` as `DT.String` (labels left for the target), `user` as nominal (four participants), `gender` as binary, and sixteen sensor or body-size intervals (`age`, `height`, `weight`, `BMI`, `x1`-`z4`). That is coherent for the CSV layout. It does not exactly match the assignment PDF's predictor inventory (see shortcomings).

The kitchen-sink forest nearly memorizes; the tuned forest trades accuracy for regularization

Step 3 fits a default `RandomForestClassifier` (100 trees, unlimited depth, minimum leaf 1) on all encoded predictors. On the **full** sample it records **accuracy 1.0000** and **0** misclassifications across all five classes. That is an overfitting diagnostic, not a deployable model.

On a **stratified 70/30** holdout (115,943 train / 49,690 validation, `random_state=31415`), the same defaults give **training accuracy 1.0000** and **validation accuracy 0.9966** (170 validation errors, **0.3%** MISC). Validation is weakest on `standingup` (**1.9%** class MISC) and `sittingdown` (**1.0%**); `sitting` and `walking` are essentially clean. Top importances are sensor axes (`z1` 0.144, `z2` 0.123, `y3` 0.110, `y2` 0.103, `x4` 0.080), not demographics.

Steps 4-6 narrow features, search hyperparameters on a **10%** stratified optimization sample, then score the winner on a fresh **70/30** holdout and a **K=2…10** cross-validation sweep.

Feature selection and the optimization sample

A single default decision tree on **all** rows ranks importances; keeping features until **99%** cumulative importance yields **21 -> 16** predictors. Demographics and user dummies sit at the bottom (`gender` 0.0033 cumulative 0.9949; `user` columns under 0.002 each). The cut drops `height` and two user levels but keeps `age`, `weight`, `BMI`, and `user3:jose_carlo` at zero importance in the later optimized forest.

OPTIMIZATION_SAMPLE = 0.1 draws **16,563** cases (stratified on `activity`) for grid search. The search runs **3,000** parameter combinations with **4-fold** CV (**12,000** fits), completing in **32 min. 31 sec.** (~0.65 s per combination).

Leaf and split grids are anchored at **m = floor(0.005 x N) - (classes - 1)** on the full **N = 165,633**, so **m = 824** and `min_samples_leaf` centers on **824** with `min_samples_split` near **1652**. That is **0.5% of the full dataset** per leaf, even though tuning sees only 10% of the rows. The winning specification is:

Parameter	Value
criterion	gini
max_depth	7
max_features	4
min_samples_leaf	824
min_samples_split	1652
n_estimators	50

On the **optimization sample only**, 4-fold CV reports **training accuracy 0.7901** and **validation accuracy 0.7893** (misclassification ratio **1.0035**). Optimized importances are again dominated by **z2**, **z1**, **y2**, **y3**, **x4**.

Holdout on the full data with selected features

Refitting that forest on **70%** of all rows (same seed) gives **training accuracy 0.9159** and **validation accuracy 0.9132** (misclassification ratio **1.0322**). Overall validation MISC is **8.7%**-far above the kitchen sink's **0.3%** on the same split geometry but a different model.

The error is **not** spread evenly across activities:

Activity	Validation class MISC (tuned)	Validation recall (tuned)
sitting	0.4%	1.00
sittingdown	31.1%	0.69
standing	2.1%	0.98
standingup	57.6%	0.42
walking	5.5%	0.95

`sitting` and `standing`/`walking` remain strong. **`sittingdown`** and **`standingup`**-transient, posture-change classes-absorb most of the tuned model's mistakes. The validation confusion matrix shows large off-diagonal mass between `sittingdown`, `standingup`, and `standing` (e.g. 450 `sittingdown` rows predicted as `standing`, 755 `standingup` as `standing`).

So the substantive story is: **unrestricted forests fit the sensor manifold almost perfectly**; **the tuned forest the assignment selects is heavily regularized and sacrifices roughly eight percentage points of holdout accuracy** (kitchen sink **99.66%** vs tuned **91.32%**) in exchange for finite leaves and a misclassification ratio near **1.03** on that single holdout. Whether that trade is desirable depends on whether the huge leaf constraint actually fixes a generalization problem or only underfits transition classes.

Optimization-sample CV vs full holdout

There is a **~12-point gap** between grid CV on **16,563** cases (**~78.9%** accuracy) and the **70/30** holdout on **165,633** rows (**~91.3%** validation). That gap is expected in part: the same hyperparameters are evaluated on much more training data at holdout time, and the optimization fold sees only a tenth of the corpus. It still means **the reported "optimum" metrics during tuning understate the holdout score** and **overstate error** relative to the final refit. The write-up should treat **0.7893** as a pessimistic, small-sample CV number, not as the model's operational accuracy.

K-fold sweep: rising validation error at low K and a high ratio at K = 10

With tuned hyperparameters fixed, stratified **K** from **2** to **10** shows **test misclassification** falling from **0.3010** (2-fold) to **0.1403** (10-fold), while **train MISC** stays near **0.06-0.08** only at low **K** (2-fold train **0.0625**) and rises toward **0.0793** at 10-fold. **Mean misclassification ratios** fall from **4.82** (2-fold) to **1.77** (10-fold) but remain **well above** the **1.2** comfort threshold used elsewhere in the course scripts.

The run labels **10-fold** "best" because it has the highest mean test accuracy (**0.8597** vs holdout **0.9132**). That **~5.4-point** gap between 10-fold CV and the single holdout, combined with **test MISC ±0.0846** at K = 10, suggests **fold-to-fold variance** and possibly **structure in how rows cluster by user or session** that plain stratified folds do not break. At **K = 2**, **test MISC ±0.1547** and ratio **±3.67** are extreme: half the data in each test fold makes validation unstable for rare classes like `sittingdown` and `standingup`.

The summary block prints **accuracy** ratio **1.0710** at K = 10 but **misclassification** ratio **1.7700**-consistent with train accuracy **0.9207** and validation **0.8597**. The holdout, by contrast, shows train and validation accuracies almost tied (**1.0030** accuracy ratio). **K-fold therefore paints a more pessimistic generalization picture than the one holdout split**, especially for the misclassification ratio.

---

2. What worked well

The solution follows the Week 7 nominal-forest workflow end to end: documented data map, RIE, kitchen-sink reference, tree-based feature selection, stratified case reduction for tuning, `GridSearchCV`, holdout validation, and a K-fold sweep. Seeds are fixed (**31415**), splits are stratified on `activity`, and the target is excluded from imputation via `no_impute` when typed as `DT.String`. Those choices match the template and show up clearly in `wk7_output.txt`.

The **kitchen-sink vs tuned** contrast is the pedagogical center of the run, and the output makes it visible: perfect in-sample fit, **99.66%** holdout for defaults, then **91.32%** holdout with **min_samples_leaf 824** and depth **7**. The regularization is not cosmetic; class-wise tables show where accuracy went (transition activities).

Feature selection is transparent: the tree-importance table, the **99%** cutoff line, and the **21 -> 16** reduction are printed in full. Hyperparameter search documents the grid size, runtime, and winning settings. Importance lists before and after tuning consistently tell a **sensor-first** story (`z2`, `z1`, `y3`, `y2`, `x4`), which fits the problem statement in `Notes/Initial_Prompt.txt` (infer activity from phone sensors in a building).

Using **OPTIMIZATION_SAMPLE = 0.1** is appropriate for a **3,000 x 4**-fold search on **165k** rows; the log states the sample size (**16,563**) and keeps stratification on the target, as the PDF describes.

Arithmetic in the holdout and CV sections is internally consistent: confusion matrices align with printed accuracy, precision, recall, and class MISC percentages.

---

3. Shortcomings and limitations

Assignment PDF text is wrong in two places. Page 1 still says the exercise is to "build an ML model **(decision tree)** to identify customers likely to **default on their loan payments**"-clear paste-over from a credit-default week. It also lists "**1 binary, 2 nominal and 16 interval**" features for **165,633** readings. The implemented map and `Initial_Prompt.txt` agree on **one binary (`gender`), one nominal predictor (`user`), sixteen interval sensors/demographics, and a separate nominal/String target (`activity`)**-eighteen predictors plus target in the raw file, not "two nominal" predictors. The solution does not call out or reconcile that mismatch.

Kitchen-sink feature matrix follows the course template, not the binary/nominal templates. `X = encoded_df` with `y = df[target]` mirrors `Nominal_RandomForest_Cellphone.py`, with

`encoded_df.drop(target)` commented out. RIE treats `DT.String` activity as excluded from encoded predictors, so this is probably not literal target leakage, but it is easy to misread and differs from every binary-target template that drops the target column explicitly.

Feature selection and hyperparameter search use information from the full corpus before the final holdout. The importance tree and the **10%** optimization draw are fit on all **165,633** rows (or all selected features from that set). Only Step 5's holdout is a clean split for the reported **91.32%** score. Rows that later appear in validation influenced which features and hyperparameters were chosen. A nested split (select and tune inside train only) would be the stricter design.

Leaf and split floors are calibrated to full N while tuning on 10% of rows. `min_samples_leaf ~ 824` means each leaf must cover ~**828** training rows in the **16,563**-case optimization set-already ~**5%** of that sample per leaf, before averaging over trees. On the full **115,943** training rows at holdout refit, the same hyperparameter allows much richer partitions. That asymmetry helps explain why **optimization CV (~79%)** looks much worse than **holdout (~91%)** and makes the "optimum" on the small sample a noisy proxy for full-data behavior.

The tuned model underperforms the kitchen sink on the same validation fold geometry by design, but the assignment narrative could misread that as "improvement." Validation accuracy drops from **99.66%** to **91.32%**. The gain is regularization and interpretability of constraints (depth 7, fifty trees, large leaves), not higher accuracy. Transition classes **`sittingdown`** (**31.1%** MISC) and **`standingup`** (**57.6%** MISC) are the main casualties.

Demographics and user ID stay in the model with negligible gain. Tree selection keeps `age`, `weight`, `BMI`, and encoded `user` columns despite cumulative importance below **1%** for gender and most user dummies. For deployment to **new** building occupants, **`user` is a leakage-like surrogate** (four named participants in the map). The output never asks whether `user`, `gender`, `age`, `height`, and `weight` should be **`DT.Ignore`** so the forest must generalize from sensors alone-a natural question for "activity in a building" rather than "which of these four people is moving."

K-fold results contradict the comforting holdout ratio. Holdout misclassification ratio **1.03** suggests stable generalization; **10-fold** misclassification ratio **1.77** and validation accuracy **85.97%** do not. Low **K** blows up test error (**30.1%** at 2-fold). The script crowns **10-fold** as "Best K" by highest mean test accuracy even though that accuracy is still **5+ points below** the single holdout and the ratio remains red. The sweep is useful as a **stability warning**, not as confirmation that the model is uniformly solid.

No baseline beyond kitchen sink. There is no majority-class or sensor-only benchmark, no per-user holdout (e.g. leave-one-user-out), and no discussion of class imbalance (`sitting` **50,631** vs `sittingdown` **11,827** on the full fit). Macro vs weighted metrics differ on the tuned holdout (macro avg recall **0.81**), but the narrative stays accuracy-first.

Plots are requested but not usable in the saved log. `display_importance(..., plot=True)` triggers a non-interactive backend warning; no image paths are recorded in `wk7_output.txt`.

Variable reuse in Step 4.2 overwrites `X_train` / `y_train` from the kitchen-sink split with the **10%** optimization sample. The final holdout correctly uses `Xt` / `Xv`, but the flow is easy to misread when auditing leakage.

---

4. Recommendations

1. **Fix the data dictionary in the write-up.** State explicitly: target `activity` (five classes); predictors **gender** (binary), **user** (nominal, four levels), and **sixteen** interval fields. Note the PDF's "loan default / decision tree" sentence and "2 nominal" feature count as errors so graders are not confused.
2. **Drop or ignore demographics for a "new user" story.** Refit the tuned specification with `user`, `gender`, `age`, `height`, `weight`, and `BMI` set to `DT.Ignore` (or excluded after selection) and compare holdout accuracy and **`standingup` / `sittingdown`** recall. If sensor-only performance is within a point or two of **91%**, prefer that

model for the stated building use case.

3. **Nest feature selection and grid search inside the training fold.** On the same **70/30** split used for final reporting, run the **99%** importance tree and the **3,000**-combo search only on train (optionally the inner **10%** subsample of train). Report validation once on the locked **30%**.
4. **Align leaf/split grids with the optimization sample size.** Either compute **m** from `0.005 x len(optimization_sample)` or scale leaf bounds when refitting on full train so CV on **16,563** rows reflects the same capacity as the final forest.
5. **Treat K-fold as a diagnostic, not a second "best score."** Report the band from **K = 2** (**30.1%** test MISC) to **K = 10** (**14.0%** test MISC) and the **1.77** ratio at K = 10. Add **GroupKFold** or **leave-one-user-out** CV to test whether fold instability reflects the four users repeating thousands of correlated rows.
6. **Lead conclusions with transition-class metrics.** For operational use, report recall and MISC for **`sittingdown`** and **`standingup`** on holdout (currently **0.69** and **0.42** recall). Consider class weights or a hierarchical model (static vs transition) if those activities matter.
7. **Clarify the kitchen-sink vs tuned comparison in plain language.** Kitchen sink: **100% / 99.66%** train/val accuracy, unbounded depth. Tuned: **91.59% / 91.32%**, ratio **1.03**, with large leaves. The tuned model is **more conservative**, not more accurate on this split.
8. **Save importances and tree plots to files** referenced in the text log so `wk7_review` deliverables stand without rerunning.

---

5. Overall opinion

This is a **complete Week 7 random-forest pipeline** with a clear sensor-driven signal and honest documentation of runtime-heavy tuning (**~32 minutes**, **3,000** combinations). The unrestricted forest shows what unlimited capacity does on **165k** correlated readings (**100%** training, **99.66%** validation). The selected forest-**Gini**, depth **7**, **50** trees, **824**-sample leaves, **four** features per split-lands near **91%** holdout accuracy with a **1.03** misclassification ratio on one split, while **10-fold** cross-validation at the same hyperparameters reports only **~86%** validation accuracy and a **1.77** misclassification ratio. The most important substantive finding is class-specific: **static activities are easy; `standingup` and `sittingdown` are not**, with roughly **42%** and **69%** recall on holdout after tuning.

What should be tightened is interpretation, not necessarily another hour of grid search. The assignment PDF's loan-default wording and feature count should be corrected in the narrative; **`user` and body demographics** should be justified or removed for generalization; **optimization-sample CV (~79%)** should not be read as the same quantity as **holdout (~91%)**; and the **K-fold sweep** should be read as evidence of **fold instability** (especially at low **K**), not as a second green light next to the holdout. The fitted tuned model is a **reasonable, heavily regularized baseline** for the course exercise; the kitchen-sink **99.66%** line is the more optimistic bound, and the transition-class table is where a building-audit use case would live or die.